

Universidad Rafael Landívar.

Facultad de Ingeniería

Ingeniería en Informática y Sistemas.

Curso: Manejo e Implementación de Archivos | Sección 2

Docente: Ing. Luis Enrique Aguilar Rojas



ACTIVIDAD 1 - SEMANA 13 – API CONCEPTOS

Estudiante

Fátima Joanna López Quiñonez - 1088825

Guatemala, 1 de octubre de 2026

APIs conceptos

Una API (Application Programming Interface) es un contrato que permite que un software acceda a datos o utilice funciones que ofrece otro software. La API define qué se puede solicitar y qué información puede devolverse, mientras oculta los detalles internos del sistema.

A) Tipos de API

APIs públicas: Son APIs disponibles para cualquier desarrollador u organización a través de internet. Generalmente tienen pocas restricciones, aunque pueden requerir una API key. Permite integraciones y desarrollos de terceros.

- Ejemplo: PayPal ofrece APIs REST que permiten integrar funciones de pagos, órdenes, facturación y suscripciones en otras aplicaciones.

APIs internas: Son APIs que no están disponibles públicamente y se utilizan dentro de una organización. Requieren autenticación y sirven para conectar diferentes sistemas internos.

- Ejemplo: Netflix utiliza una arquitectura formada por cientos de microservicios para diferentes funciones de su plataforma. Estos servicios necesitan comunicarse entre sí, por lo que representa un ejemplo de un entorno donde pueden utilizarse APIs internas para conectar servicios de la organización.

APIs de socios: Son APIs disponibles solo para un grupo de socios comerciales autorizados. Aunque pueden funcionar mediante internet, no están abiertas para cualquier persona.

- Ejemplo: Shopify Partner API. Ofrece una API para socios que permite a los usuarios autorizados acceder a información de su organización. El acceso requiere una organización y un token de acceso de partner.

APIs compuestas: Combinan varias llamadas a diferentes APIs o servicios y presentan el resultado mediante un único punto de acceso. Tiene una capa de solicitud, una de consulta de los diferentes servicios y devuelve la información en una sola respuesta.

- Ejemplo: Microsoft Graph permite utilizar solicitudes por lotes para combinar varias solicitudes en una sola llamada HTTP.

B) Estilos arquitectónicos

1) REST (Representational State Transfer): Es un estilo utilizado para APIs web. Trabaja con recursos, identificados mediante una URL, y utiliza métodos HTTP como get, post, put, patch y delete. Entre sus características están la separación entre cliente y servidor y que cada solicitud sea independiente.

- Ejemplo: `Get /api/users/45` // solicita información del usuario 45

2) GraphQL: Es un lenguaje de consulta y un entorno de ejecución que permite al cliente indicar exactamente qué información necesita. Se usa principalmente en sistemas completos donde diferentes clientes tienen diferentes necesidades de información.

- Ejemplo: Una aplicación puede solicitar solamente el nombre y correo de un usuario, en lugar de recibir todos sus datos.

3) SOAP (Simple Object Access Protocol) es un protocolo de comunicación basado en XML y utiliza estructuras y estándares estrictos. Funciona sobre distintos protocolos como HTTP y SMTP.

Se utiliza especialmente en entornos empresariales donde son importantes los estándares formales, el manejo de errores y la seguridad.

- Ejemplo: Una API de un banco podría permitir consultar el saldo de una cuenta. El usuario envía su número de cuenta y la API procesa la solicitud para devolver el saldo disponible.

4) gRPC: es una tecnología que permite que diferentes aplicaciones o servicios se comuniquen entre sí de forma rápida. Usa Protocol Buffers, que permite enviar los datos de manera más compacta. Se utiliza principalmente para conectar servicios dentro de una misma aplicación, cuando se necesita una comunicación rápida y constante.

5) Web Socket / Webhooks

- los Web Sockets mantienen una conexión persistente y bidireccional entre cliente y servidor. Son útiles cuando se necesita recibir información en tiempo real, como en transmisores de datos.
- los Webhooks funcionan de manera distinta. el servidor envía una notificación al cliente cuando ocurre un evento determinado, sin que el cliente tenga que solicitarla.

6) Conceptos relacionados

- Payloads: Es la información que se envía entre el cliente y el servidor. puede estar en formatos como json, XML o datos binarios.

- Ejemplo:

```
{  
  "bookId": 81,  
  "title": "Harry Potter"  
}
```

En una solicitud POST, estos datos pueden representar la información de un nuevo libro

- Request: Un request o solicitud es el mensaje que una aplicación envía a un servidor para pedir información o realizar una acción.

Contiene una URL, método HTTP, headers y un body.

- Ejemplo: Get /api/users/45 (solicita información del usuario 45)

- Response: Es el mensaje que el servidor devuelve después de recibir una solicitud. Está compuesto por un código de estado, headers, y un body con información.

- Ejemplo:

```
{  
  "id": 45  
  "nombre": "Oscar"  
  "edad": 24  
}
```

El servidor recibe la solicitud para consultar al usuario 45 y responde con su información.

Algunos códigos de respuesta son:

[200 OK] : solicitud exitosa

[201 Created] : se creó un nuevo recurso

[404 Not found] : NO se encontró el recurso

[500 Internal Server Error] : Ocurrió un error de servidor

→ CURL (Client URL) es una herramienta de línea de comandos que permite enviar y recibir datos desde un servidor. Es útil para comprobar endpoints y comprobar si la API funciona correctamente.

• Ejemplo: `curl http://ejemplo.com`

también se puede usar en métodos como get, post, put y delete.

→ Tipos de autorización en Postman:

• No auth: NO se envía información de autorización

• API key: se envía una clave por medio de los headers o los parámetros de la URL.

• Bearer token: Utiliza un token de acceso enviado en el header.

JWT Bearer: Utiliza JSON Web Token, que contiene información codificada y firmada.

• Basic Auth: Utiliza un nombre de usuario y una contraseña.